

Metadata Spoofing Attacks in Federated Learning: A Vulnerability Analysis

Amin Alhawary (21041123), Mohamed Elafifi (22066939), Sonia Khatun (24051725)

Abstract <<100 –150 words>>

Keywords <<5 Keywords from:

Federated Learning, Decentralised Learning, Heterogeneous Federated Learning, Clustered Federated Learning, Knowledge Distillation, Personalised Federated Learning, Model Poisoning, Backdoor Attacks, Byzantine Clients, Model Architecture, Model Metadata
>>

Introduction

<< In this section, state the purpose and scope of your project, the rationale of the work, the novelties, the list of research questions, the structure of the rest of the report. >>

Federated Learning (FL) is a setup in machine learning, which entails the use of multiple client devices to collaboratively train models, allowing data to never leave the client devices, which makes it appealing for many applications. With a growing concern in industry over data security, use of compute resources and a desire to maximise benefit by collaboration, it makes studying FL, and by extension its vulnerabilities, vital.

FL setups can be statistically heterogeneous, where the data split is non-independent and identically distributed (Non-IID), or model heterogeneous, where different model architectures are used across client devices. To deal with statistical and model heterogeneity, Clustered Federated Learning (CFL) was introduced to groups similar clients together.

However, this system has a potential vulnerability: when assigning clients to clusters, the server may rely on information provided by clients, such as their declared model architecture or architecture-related metadata (e.g., arch). A malicious client can exploit this dependency by misrepresenting its architecture and manipulating its model update. Such manipulation may interfere with the similarity or distance calculations used by the server for cluster assignment, such as cosine similarity.

Once a malicious client is assigned to an inappropriate cluster, it may inject a malicious update, such as weight boosting or sign-flipping, which could potentially bypass traditional coordinate-wise security filters. This can negatively affect model accuracy and convergence within the affected cluster. The impact may be more significant when clients have Non-IID data distributions, where the data characteristics differ across clients and clusters.

Therefore, this research aims to investigate the vulnerability of the server-side cluster assignment mechanism to architecture spoofing attacks and to evaluate the

effectiveness of a shape-agnostic, duck-type defence pipeline in detecting and filtering architecture-level manipulation. Based on this problem, the following research questions are proposed:

RQ1	Could falsely declaring a client's metadata (including architecture and data distribution) degrade its target model accuracy?
RQ2	What factors in architecture and in data distribution can allow such metadata spoofing attacks to cause more or less damage?
RQ3	How significantly does targeted cluster misallocation degrade model convergence and classification accuracy across Non-IID data distribution?
RQ4	
RQ5	

RQ1: How susceptible are Clustered federated learning frameworks to architecture spoofing attacks during client cluster assignment?

RQ2: To what extent does misrepresenting declared architecture metadata force improper cluster routing, and does the routing mechanism validate this metadata against the submitted model update?

RQ3: How significantly does targeted cluster misallocation degrade model convergence and classification accuracy across Non-IID data distribution?

The main contributions of this work are:

- A detailed

Problem statement -> research gap (briefly)

Research Questions

Contributions (novelty?)

Relevant Work

<< This section is to include the literature review of very relevant works including the gap analysis that highlights the gaps and how you relate to your research questions >>

Federated Learning, Heterogeneous Federated Learning

Clients in Federated Learning (FL) are heterogeneous by design, as they inherently differ in their local data distributions, computational capabilities, and underlying model architectures. To address model heterogeneity, several paradigms have been proposed, including Personalised Federated Learning, Knowledge-Distillation-Based Federated Learning, and Clustered Federated Learning (CFL). Because raw model weights cannot be directly **aggregated** across differing architectures, specialised CFL frameworks cluster clients using alternative uniform representations, such as local model outputs on a shared public dataset or specific personalisation layer parameters (Vinyals et al., 2015; Li et al., 2019; Arivazhagan et al., 2019).

Knowledge distillation (Vinyals et al., 2015) transfers learned representations from a larger teacher model to a smaller student model to facilitate cross-architecture learning. Extending this concept to FL, FedMD (Li et al., 2019) enables server-side knowledge sharing and aggregation without requiring structural uniformity, allowing each client to independently design its local model architecture. Experimental evaluations on subsets of the FEMNIST, CIFAR-10, and CIFAR-100 datasets demonstrate that this collaborative framework achieves substantial performance gains across participating clients. Complementary to knowledge-sharing frameworks, Arivazhagan et al. (2019) introduced FedPer, which embeds client-specific personalisation layers into each local model while maintaining a shared base architecture to mitigate the impact of Non-IID data distributions.

In addition to data-driven grouping, architecture-based clustering has emerged as a key strategy within CFL to manage model heterogeneity. Under architecture-based clustering, the central server assigns clients into distinct sub-clusters based on shared structural properties—such as layer count, parameter dimensions, or network depth—allowing clients with identical or compatible architectures to aggregate weights directly within their assigned group. However, because central aggregation servers typically rely on unverified client declarations to determine these architectural groupings, this design creates a novel security vulnerability. Operating across structurally irregular system components and trusting self-reported metadata leaves federated networks increasingly susceptible to architecture spoofing, where malicious actors falsify their structural characteristics to manipulate cluster assignment.

Adversarial and Poisoning Attacks in Federated Learning

Because the central server cannot inspect local training data due to privacy-preservation requirements, this design creates a fundamental security weakness. These adversarial threats operate primarily through data poisoning and model poisoning. Bhagoji et al. (2019) demonstrated that under a white-box threat model, an attacker controlling as few as 1% to 5% of clients can directly manipulate gradient updates to achieve over a 90% targeted attack success rate while maintaining near-baseline main-task accuracy—making it exceptionally difficult to detect via standard validation metrics.

Poisoning attacks are also divided into untargeted attacks and targeted attacks. Untargeted attacks degrade the overall performance of the global model by causing incorrect predictions across many input classes, while targeted attacks — including backdoor attacks — are more difficult to detect, since the model performs normally on regular inputs but produces an attacker-controlled prediction only when a specific trigger pattern is present (Bagdasaryan et al., 2020). As a result, a single malicious update may dominate the aggregation process and significantly corrupt the global model. Therefore, identifying these updates is crucial to ensure the integrity of the

global model. However, this remains difficult because malicious updates are mixed in with updates from many honest participants.

This difficulty is further compounded by the Non-IID data problem (Li et al., 2020): data collected from different clients naturally varies due to differences in local environments and distributions. For example, clinical datasets from different hospitals may vary because of differences in patient populations, medical practice, and equipment calibration. As a result, the server cannot easily determine whether an unusual update reflects natural data variation or a malicious attack, allowing attackers to exploit this statistical heterogeneity to hide among honest clients.

While data poisoning, model poisoning, and backdoor attacks are well studied in traditional FL, security during CFL's clustering stage remains largely unexplored — particularly the risk of malicious clients misrepresenting their hardware, resources, or architecture to manipulate cluster assignment and gain unauthorized access to a target cluster, since entry into the cluster is the objective in itself: it grants access to the specialised model regardless of whether a poisoning payload survives aggregation..

Consequently, this dissertation focuses on analysing architecture spoofing attacks within Clustered Federated Learning to evaluate their impact on cluster stability and model robustness. The selection of these two evaluation dimensions is academically justified by foundational frameworks in adversarial Clustered Federated Learning (CFL). First, tracking cluster stability via geometric partitioning metrics is required to quantify whether the server can maintain client grouping purity against unauthorized intruders (Sattler et al., 2020). Second, evaluating model robustness using Attack Success Rate (ASR) and Robust Accuracy — standard metrics in poisoning-attack literature (Bhagoji et al., 2019) — measures the local cluster's capacity to neutralize malicious gradient injections if a perimeter breach occurs. Driven by these requirements, this study addresses the following core problem:

Metadata and Behavioural Fingerprinting

In heterogeneous Federated Learning (FL), client model metadata—such as architecture type, depth, and total parameter count—is often used to determine how clients are grouped or aggregated. However, this metadata is generally accepted by the server without verification. A body of hardware and side-channel security research shows that model architectures leave detectable execution signatures even outside the model itself — through cache-timing patterns (Yan, Fletcher and Torrellas, 2020), memory-access and kernel-latency behaviour (Hu et al., 2020), power consumption (Gao et al., 2024), and electromagnetic emissions (Batina et al., 2019) — establishing that false architecture declarations are, in principle, detectable.

More directly relevant to this work, a parallel line of research shows that a model's *behaviour* — rather than its physical execution trace — is itself sufficient to expose its architecture or properties. Black-box output analysis can reverse-engineer architectural details from predictions alone (Oh et al., 2018; Rolnick and Kording,

2020), and meta-classifier techniques demonstrate that raw parameter updates expose implicit model properties without requiring hardware access (Ateniese et al., 2015; Ganju et al., 2018). This behavioural approach extends naturally to FL: the FLARE framework infers client architecture families from encrypted traffic signatures using packet timing, size, and direction (Shuvo and Hossain, 2025; Shuvo et al., 2026). While these side-channel and model-based fingerprinting methods effectively identify model characteristics (Cao, Jia and Gong, 2021; Peng et al., 2022; Chen et al., 2022), evaluation metrics remain fragmented across the literature. Cache-side-channel studies focus on search-space reduction—such as reducing VGG-16 candidate configurations from over 5.4×10^{12} to 16 (Yan, Fletcher and Torrellas, 2020)—whereas sequential structure recovery relies on edit distance metrics (achieving 99.75% structure accuracy and 99.60% layer-wise hyperparameter F1-scores in Gao et al., 2024). Traffic-based classifiers evaluate F1-scores, where FLARE yields over 98% in closed-world and 91% in open-world settings (Shuvo et al., 2026). Conversely, model ownership studies evaluate ROC-AUC alongside matching rate, robustness, and uniqueness (Cao, Jia and Gong, 2021; Peng et al., 2022). Crucially, none of these existing paradigms evaluate binary identity verification, which checks whether an observed execution fingerprint matches a client's declared metadata rather than inferring what the underlying architecture is.

Research Gap

While extensive research exists on heterogeneous FL, poisoning attacks, and architecture inference, a significant research gap remains: no existing literature investigates deploying behavioural or architectural fingerprinting during the initial cluster assignment stage of Clustered Federated Learning to verify declared client metadata. Prior CFL research assumes client metadata is inherently trustworthy or relies solely on gradient similarities after clustering has occurred. Meanwhile, side-channel fingerprinting research treats architecture recovery strictly as a passive reconnaissance task ("What model is running?") or an ownership verification task, completely bypassing the threat of active architecture spoofing during server-side cluster assignment.

How This Paper Fills the Research Gap

This dissertation addresses this gap by proposing and evaluating a lightweight, fingerprint-based verification mechanism designed specifically for the cluster assignment phase in Clustered Federated Learning. Instead of passively classifying client models, the proposed framework directly compares a client's declared architectural metadata against its empirical execution fingerprint. By establishing this verification boundary before cluster grouping takes place, the paper demonstrates how verifying metadata consistency prevents unauthorized cluster entry, preserves geometric cluster stability (evaluated via Adjusted Rand Index), and mitigates downstream poisoning amplification (evaluated via Attack Success Rate and main-task accuracy).

Research Question: How does architecture spoofing during the cluster-assignment phase of Clustered Federated Learning (CFL) affect cluster stability and downstream

model robustness, and to what extent can behavioural fingerprinting verification of declared client metadata prevent unauthorised cluster entry?

3 Methods & Materials

<< In this section, you are expected to present the method you studied and the data you used including pre-processing and the sources you acquired from.

3.1 General Methodology

Weekly stand-ups were used to provide progress updates and ensure the project timeline remained reasonable. When delays would take place, there was a use of the members of the team, to tackle multiple tasks at once.

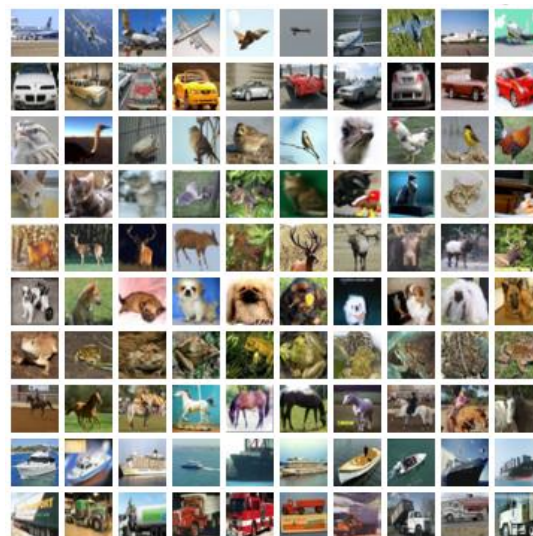
Our approach evaluates established frameworks of clustered federated learning. The initial experimental setup aimed to explore the metadata declaration gap within model-architecture-based clustering. The study was expanded to include data-distribution-based clustering configurations. For ease, we will use “Model Heterogeneity Strand” and “Statistical Heterogeneity Strand” to differentiate between both parts of the study.

3.2 Datasets

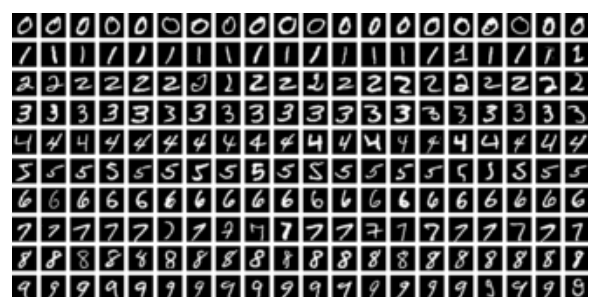
To fully study the effects of these vulnerabilities in FL, it is important to cover different problem spaces. To do so, we have chosen to study both tabular data as well as machine vision data.

For tabular data, we have chosen the IDSIoT2024 dataset. The dataset brings together real-world data for cyber-security intrusion detection. The exact version used is one previously cleaned by our co-supervisor Aimen Djemaa, in some of his research.

For the MV problem, CIFAR-10 was originally chosen, as it is widely used in FL literature as a baseline. CIFAR-10.



However, due to its high computing costs, a less demanding dataset was chosen: MNIST. MNIST is a simpler dataset consisting of.



This dataset is still used within FL, giving our study more relevance. Some runs performed on CIFAR-10 were kept, to provide context for some MV results.

3.3 Setup

3.3.1 Clustering Model

The clustering setup chosen employs 20 clients, equally split among two clusters. As mentioned, each strand of experiments will utilise a different form of clustering.

The following assumptions were taken for both strands:

- No network communication drop-offs are modelled, as the simulation executes within a unified hardware environment.
- The attacker has knowledge of the FL setup, as well as the architectures used.
- Clustering is based on manual declaration, which the server naively assumes is truthful

For each dataset, a selection of hyperparameters was chosen, based on manual calibration to reach baseline results. IDS dataset used: 20 rounds of 1 epoch on an MLP, MNIST used: 40 rounds of 3 epochs, the few CIFAR-10 runs used: 80 rounds, 30 epochs.

The aggregation method used throughout the experiment is FedAvg, which just calculates the mean of the weights. The client then adopt the average weights from the previous round as its current weights in the new round.

3.3.2 Threat Model

3.3.2.1 Model Heterogeneity Strand

Within Model Heterogeneity, three categories of model heterogeneity are explored. Within each category is a list of pairing of two

	Category Name	Model Pairings for the IDS Test	
Category A	Weight Shape Matching Pairs		
Category B	Weight Shape Manipulated Pairs		
Category C	Weight Shape Reordered Pairs		

Category	Category Name	Model Pairings for the MNIST Test
----------	---------------	-----------------------------------

Category A	Weight Shape Matching Pairs		
Category B	Weight Shape Manipulated Pairs		
Category C	Weight Shape Reordered Pairs		

Statistical Het

Evaluation Model

The nature of this untargeted attack means that checking class-specific metrics, such as recall and confusion matrices, is out of scope for this work. Therefore, the main metric to evaluate these experiments is target model accuracy change relative to the baseline result. Some additional diagnostics were used to ensure the attack was successfully completed.

Experimental Results

<< [Introduce your experimentations, present your results via tabulating the results and/or plotting them into the figures. Then discuss your results >>](#)

Model Heterogeneity Strand Results

Discussion

Data Heterogeneity Results

This section attacks the declared label histogram the server clusters on, which has no verification process. A client lies in that field without any repercussions and trains honestly with the rest of the cluster and would submit an ordinary update. Both datasets in Method were ran end to end, 600 federations each. IDS-IoT is heavily skewed (DoS 41.55%, injection 1.33%) and MNIST near-balanced, which decides section 2.

Table 1. Configuration used for this section, addition to methods.

Setting	Value
model	dense 128-128-64, dropout 0.35, no batch normalisation, 37,639 parameters on IDS
aggregation	FedAvg, unweighted
partition	Dirichlet, alpha in 0.1, 0.5, 1.0, 5.0; 20 rounds, 1 local epoch
attackers	k in 1, 2, 4, 6, 8
seeds	42 to 51, run as two disjoint halves and pooled
arms	truthful control, oracle lie, blind lie

All statistics are per seed, df = 9, trend tests block on seed.

Figures show each arm's seeds 42-46 run while tables pool all ten, so a percentage in figures may not match ones beside it.

Does the attack harm the cluster it infiltrates?

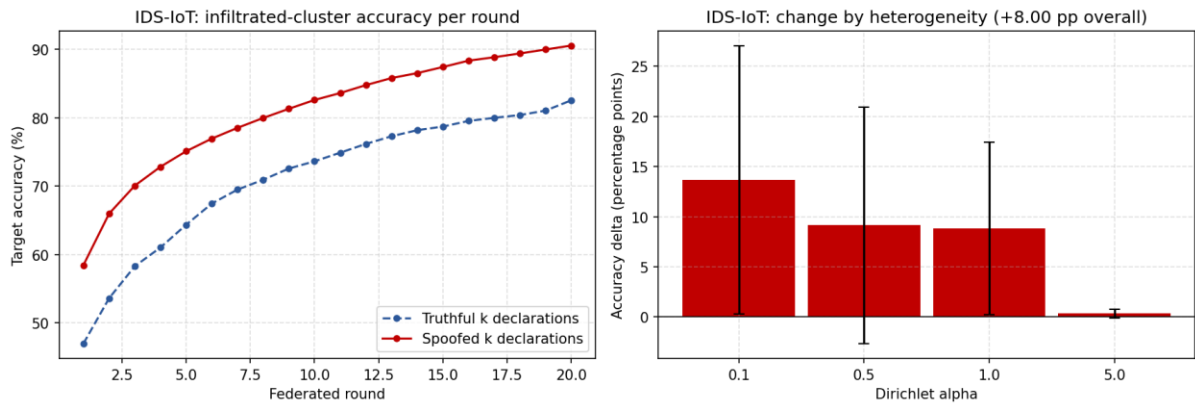


Fig. 1. IDS-IoT: the invaded cluster, per round and by heterogeneity.

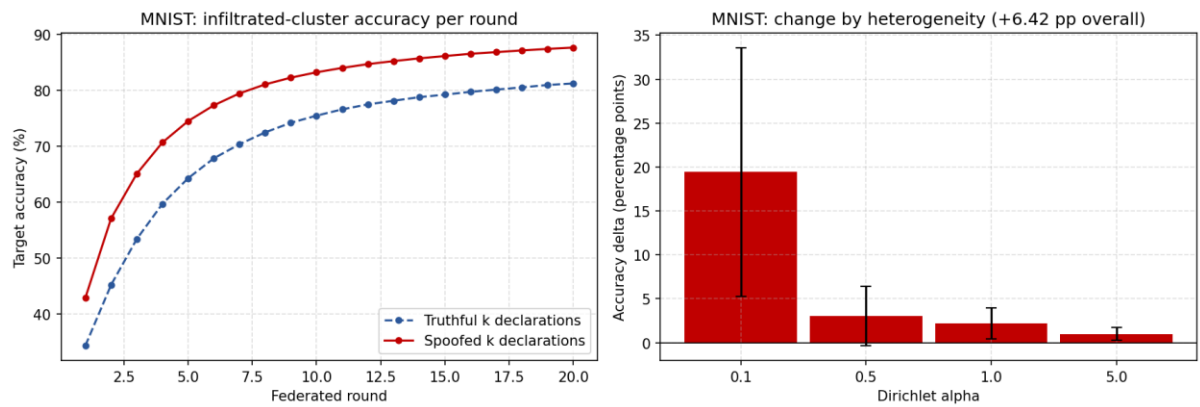


Fig. 2. MNIST: the invaded cluster, per round and by heterogeneity.

No, it helps it. The invaded cluster ends +8.00 pp better on the whole test set on IDS and +6.42 pp on MNIST, above the truthful control from the first round onward. Scored instead on its own members' data the change is +1.28 pp and -0.12 pp, so a foreign shard makes the cluster better at the whole problem while leaving it no better at its own share of it. The gain is a heterogeneity effect rather than an attack effect: +13.67 pp at alpha 0.1 falling to +0.34 pp at alpha 5.0 on IDS, and +19.44 to +0.98 pp on MNIST. Where clients already hold near-uniform label mixes the intruder supplies nothing the cluster lacks.

What does the attacker gain, and does his update stand out?

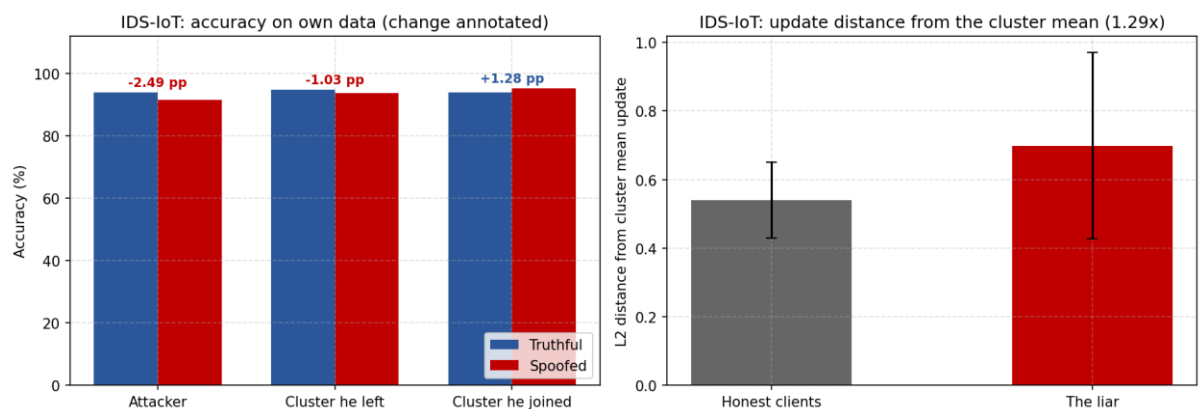


Fig. 3. IDS-IoT: attacker gain and update distance.

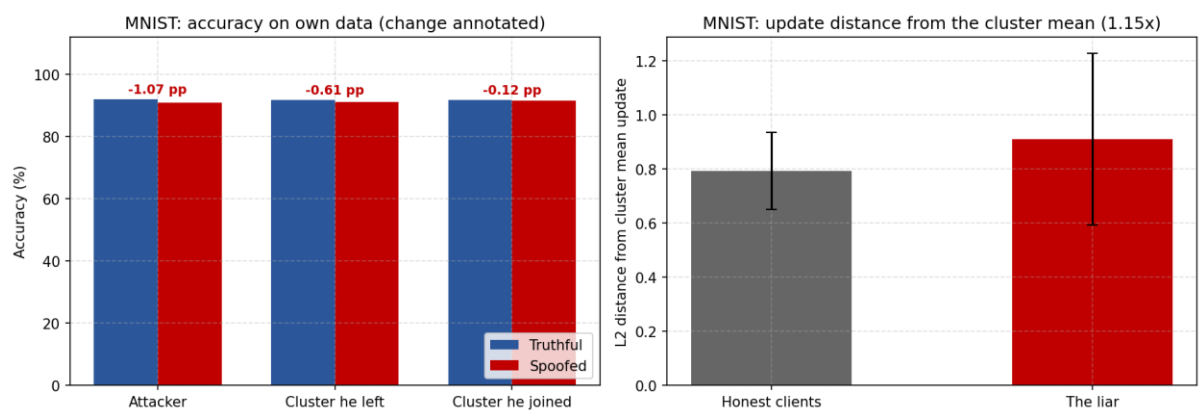


Fig. 4. MNIST: attacker gain and update distance.

He pays rather than gains: -2.49 pp on IDS and -1.07 pp on MNIST on his own data, and the cluster he abandons loses -1.03 pp and -0.61 pp. Joining a cluster whose data is unlike his own returns a model worse for his own distribution than staying home would have given. His update is also the more distant one, sitting 1.29x and 1.15x further from the cluster mean than an honest client's. The declaration is a perfect lie; the gradient is not.

Does the effect survive seeds?

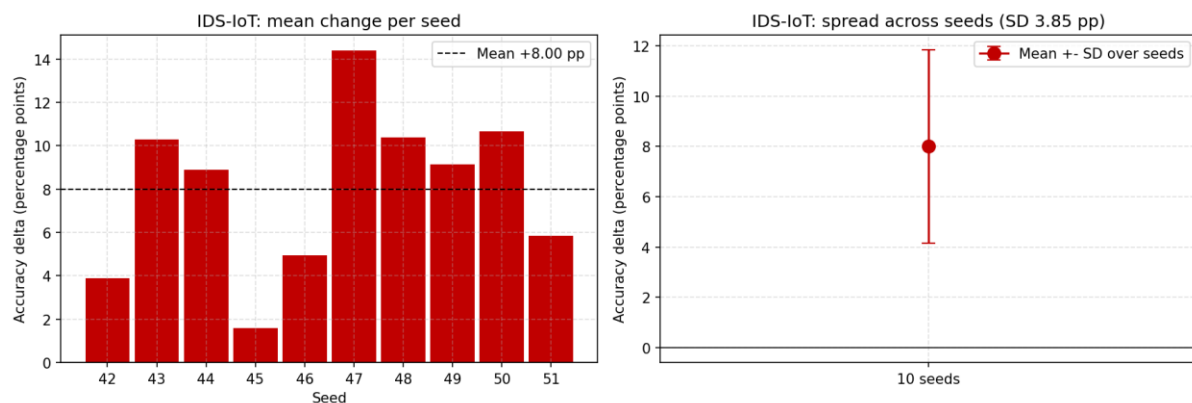


Fig. 5. IDS-IoT: the effect across ten seeds.

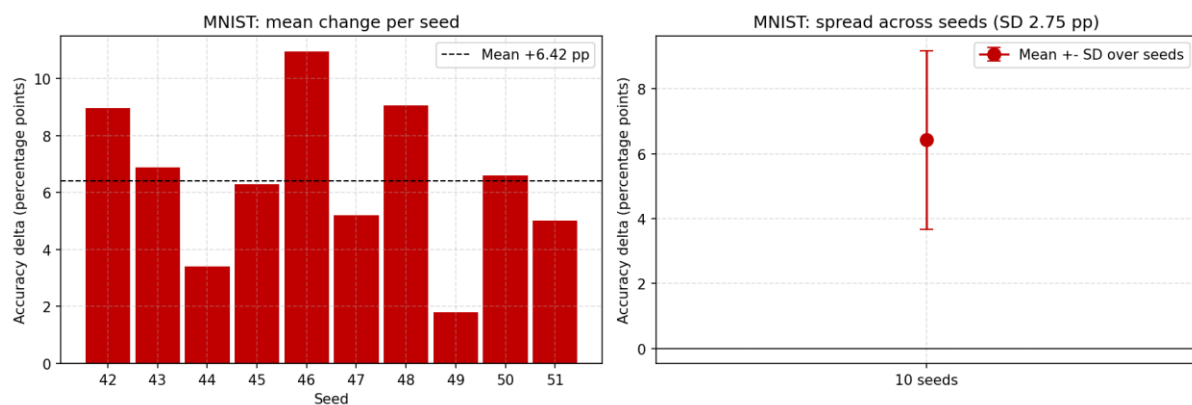


Fig. 6. MNIST: the effect across ten seeds.

Yes. 10 of 10 seeds agree on the sign in both datasets, at $+8.00 \pm 3.85$ pp and $+6.42 \pm 2.75$ pp. The across-seed spread is far smaller than the per-federation spread of 10.95 and 10.51, so nearly all the variability comes from heterogeneity rather than from randomness. With no seed reversing the sign, the rule of three puts a 95% upper bound of 3 in 10 on the rate at which a fresh seed would.

Does the effect scale with the number of attackers?

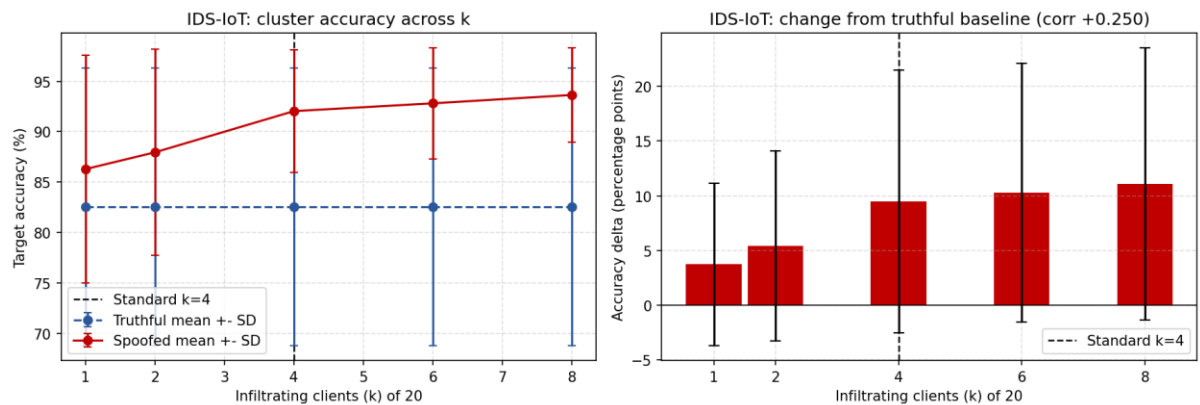


Fig. 7. IDS-IoT: the effect against attacker count.

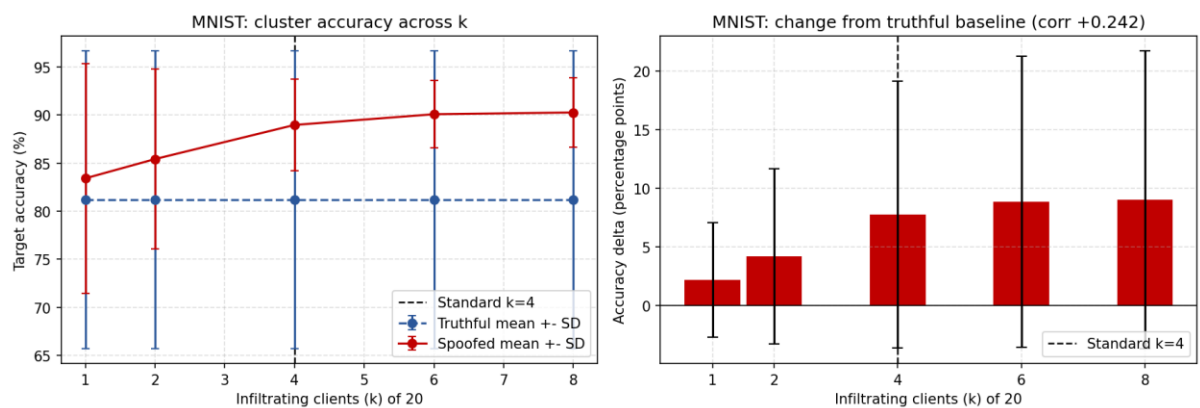


Fig. 8. MNIST: the effect against attacker count.

Yes, monotonically, and in the direction that helps the cluster: +3.74, +5.40, +9.49, +10.27, +11.10 pp for $k = 1, 2, 4, 6, 8$ on IDS, and +2.21 to +9.05 pp on MNIST, with $\text{corr}(k, \text{delta})$ of +0.250 and +0.242. At the $k=4$ budget the architecture strand uses, this channel gives +9.49 pp and +7.75 pp where that one costs its target 1.73 pp. The damage a defender watches for arrives at no budget.

Is the effect directional?



Fig. 9. IDS-IoT: the effect by which cluster was infiltrated.

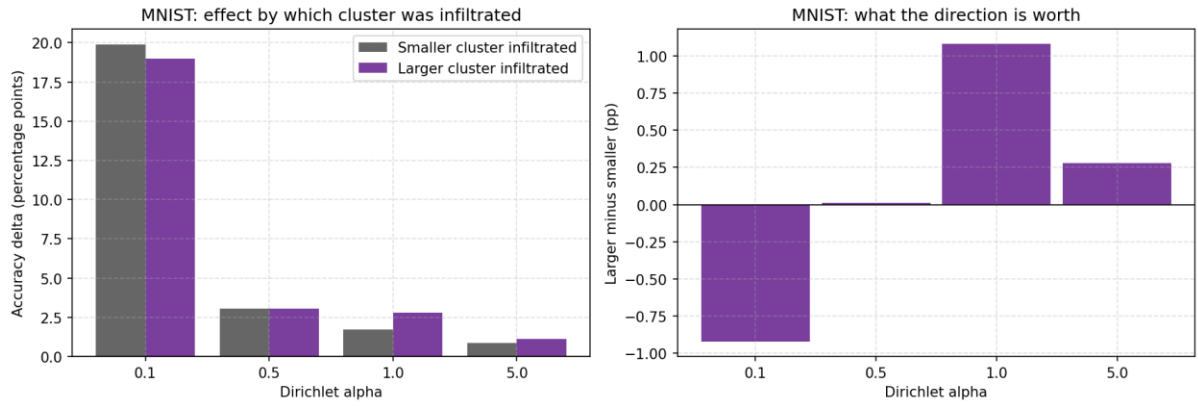


Fig. 10. MNIST: the effect by which cluster was infiltrated.

Partly. The clusters here are discovered rather than designed, so there is no pair of fixed roles to swap; instead, the runs are split by whether the larger or the smaller cluster was infiltrated, within each alpha. On IDS invading the larger cluster is worth about 7 pp more at alpha 0.1 and 0.5; on MNIST there is no directional effect at all. In all sixteen cells the invaded cluster still gains, so direction changes the magnitude and never the sign. This is an observational split rather than a designed reverse arm, which would cost 200 further federations per dataset.

Table 2. The five shared questions, ten seeds, oracle arm against its truthful control.

Measure	IDS-IoT	MNIST
invaded cluster, whole test set	+8.00 pp $\pm$ 3.85	+6.42 pp $\pm$ 2.75
invaded cluster, own members' data	+1.28 pp	-0.12 pp
attacker, own data	-2.49 pp	-1.07 pp
seeds agreeing on sign	10 of 10	10 of 10
gain at alpha 0.1 against alpha 5.0	+13.67 to +0.34 pp	+19.44 to +0.98 pp
dose-response, corr(k, delta)	+0.250	+0.242
liar's update against an honest one	1.29x	1.15x
larger against smaller cluster invaded	+6.93 pp at alpha 0.1	-0.92 pp

Membership Assignment is Inconsequentially compromised

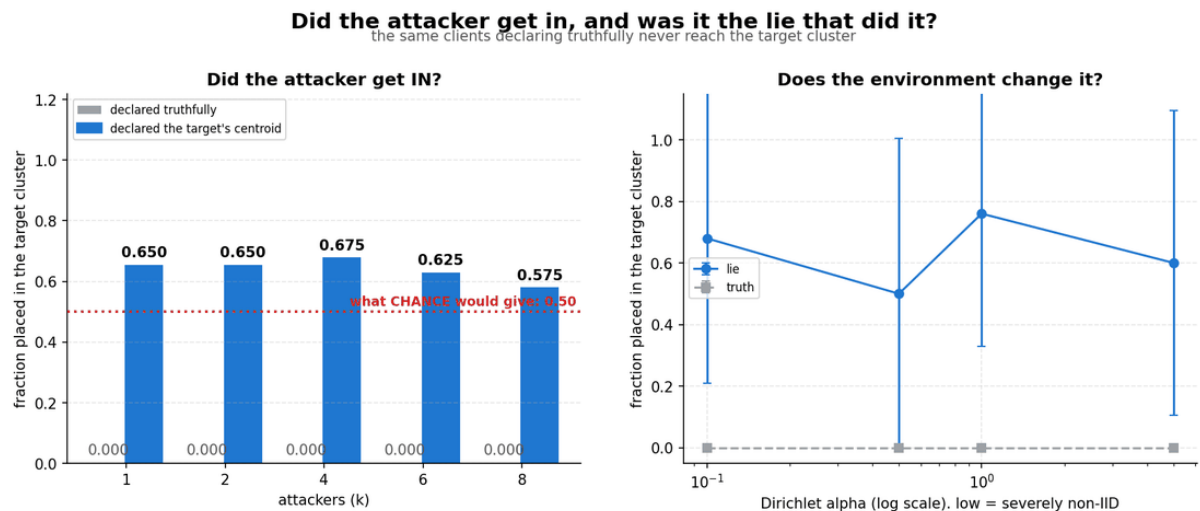


Fig. 11. IDS-IoT: Placement against attacker count.

A single client declaring its target's centroid is placed with that target in 200 of 200 federations, at every alpha and every attacker count. The truthful control never reaches the target, also 200 of 200. 0 observed failures give a 95% upper bound of 1.5% on failure rate.

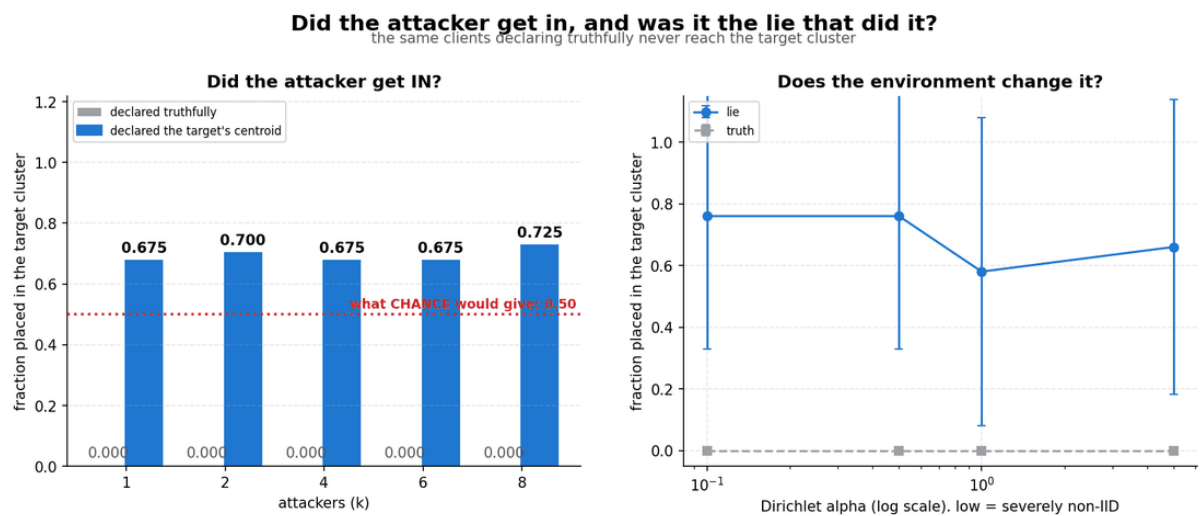


Fig. 12. MNIST: Placement against attacker count.

MNIST (Fig. 2). Placement is 0.985, with three failures in 200, at $k \geq 6$. They read 0.0 rather than a partial value. with 8 of 20 clients declaring an identical histogram, the tightest group is the attackers and the split isolates them. They become a cluster instead of joining one.

The blue bar pools the oracle and blind arms, so it sits near 0.65

Table 3. Placement and partition damage, ten seeds.

Measure	IDS-IoT	MNIST
placement, oracle	1.000 (0 failures in 200)	0.985 (3 failures)

placement, truthful control	0.000 in 200	0.000 in 200
ARI against the truthful partition	0.3229	0.3038
federations below a random partition	40 of 200	41 of 200

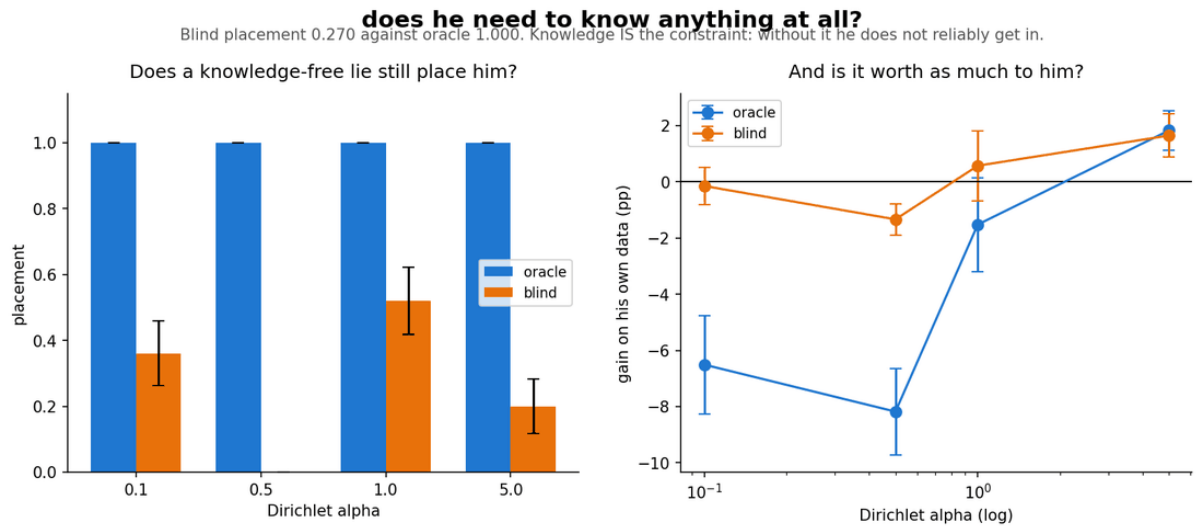


Fig. 13. IDS-IoT: blind against oracle declaration.

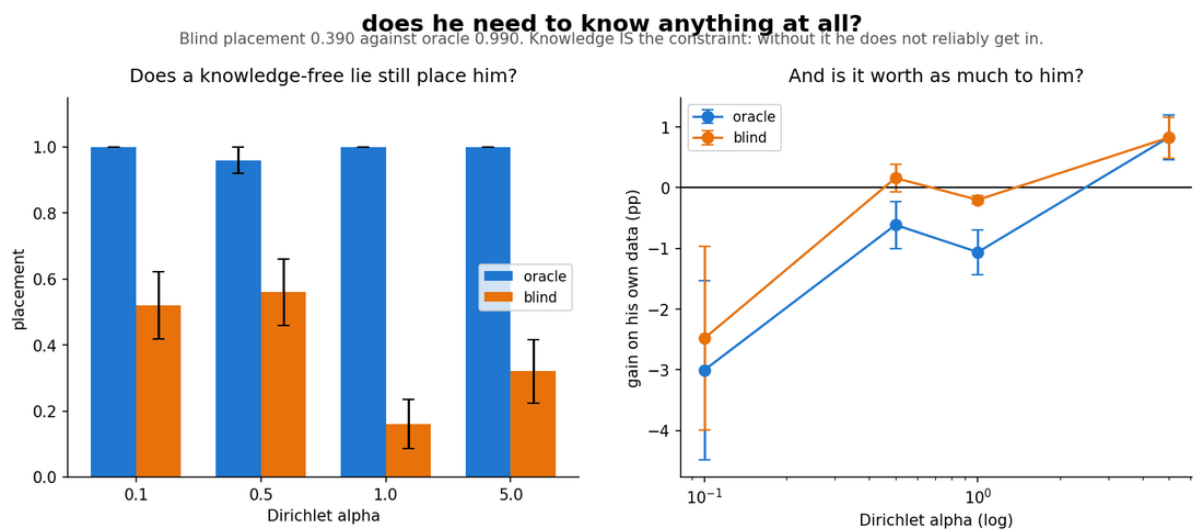


Fig. 14. MNIST: blind against oracle declaration.

Knowledge helps but is not required. A flat histogram declared with no knowledge of the target still places him 0.320 of the time on IDS and 0.430 on MNIST, with no trend across alpha. Nor is the full lie needed: 29% of the distance to the target centroid suffices on average. Every figure here therefore shows the most conspicuous version of the attack.

Infiltration costs accuracy and buys no information

Question 3: what does he learn once he is inside?

On the assumption he can actually meet -- probing his own shard -- he beats guessing in 86% of federations (85% prior-corrected). The global-pool probe reaches 99% but assumes an auxiliary set drawn from the same data as the clusters.

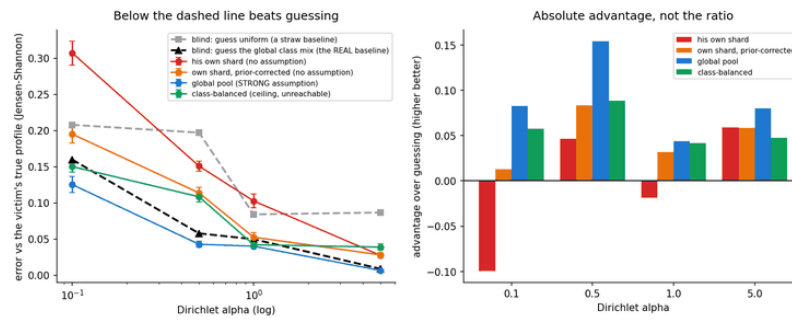


Fig. 15. IDS-IoT: the estimator ladder.

The attacker probes the victim cluster's model with data he owns and estimates its label distribution. Against a blind attacker guessing is uniform, he wins 86% of the time. That comparison is the problem; uniform is a poor guess when one class is 41.55% of the traffic. Against a blind guess of the global mix, which is public and costs no attack, he loses: 0.0964 against 0.0742, a difference of $+0.0222 \pm 0.0037$, $t = +6.03$

Question 3: what does he learn once he is inside?

On the assumption he can actually meet -- probing his own shard -- he beats guessing in 1% of federations (13% prior-corrected). The global-pool probe reaches 78% but assumes an auxiliary set drawn from the same data as the clusters.

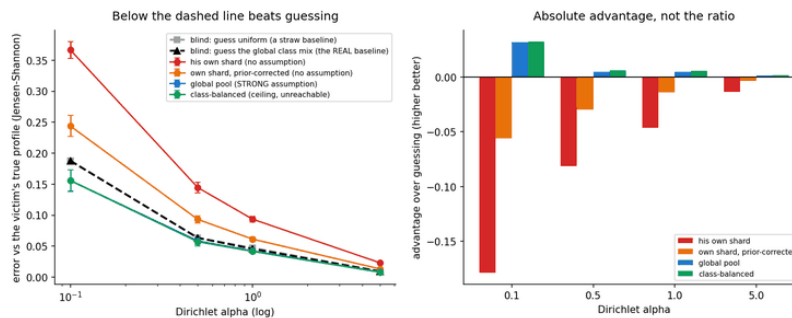


Fig. 16. MNIST: the estimator ladder.

On balanced data the global class mix is uniform, so the two dashed baselines lie on top of each other and the objection cannot survive. The negative reproduces more strongly, $+0.0235 \pm 0.0030$, $t = +7.89$, positive in all ten seeds, and he loses to the raw baseline as well, winning just 10%

Table 4. Estimation error against the victim profile, Jensen-Shannon, lower is better.

Estimator	IDS-IoT	MNIST
attacker's probe	0.0964	0.0986
guess uniform	0.1411, he wins 86%	0.0760, he wins 10%
guess the global class mix	0.0742, he wins 36%	0.0751, he wins 8%
difference, per seed	$+0.0222 \pm 0.0037$, $t = +6.03$	$+0.0235 \pm 0.0030$, $t = +7.89$
divergence, class mix against uniform	0.1224	0.0005

The attack never beat a realistic baseline on either dataset, concealed on IDS because the class skew made uniform a bad opponent.

Who pays the cost



Fig. 17. IDS-IoT: what the victim cluster loses.

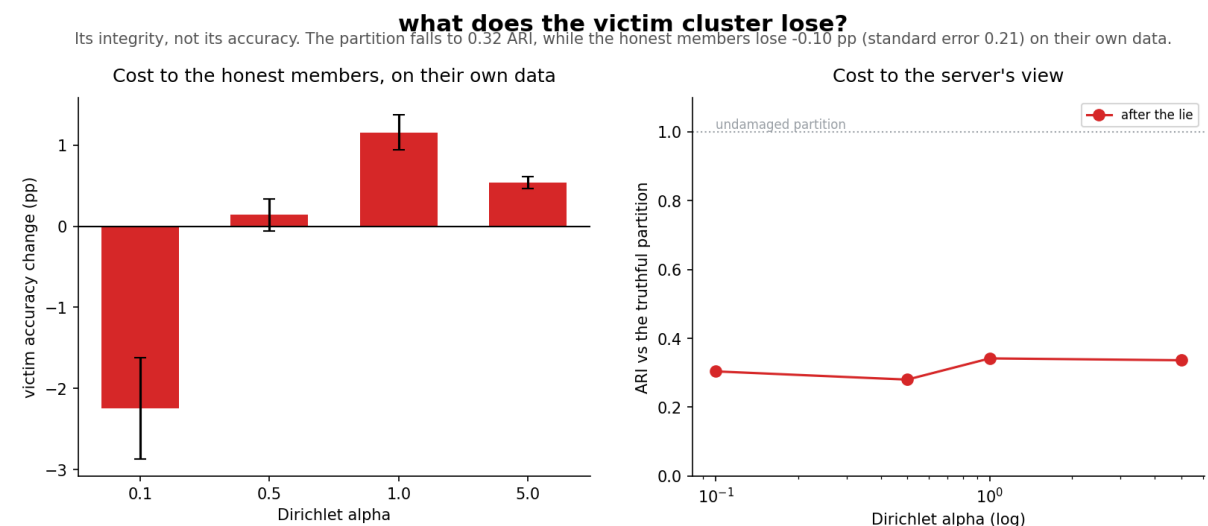


Fig. 18. MNIST: what the victim cluster loses.

The invaded cluster does not lose; on IDS it gains. It absorbs the attacker's real gradients, growing from 7.38 to 11.63 members, and improves by +1.28pp, $t = +2.76$.

what does the cluster he abandoned lose?
-1.60 pp, standard error 0.37, on the home cluster's own distribution over 100 paired cells. The attack has a second victim that no measurement so far had looked at.

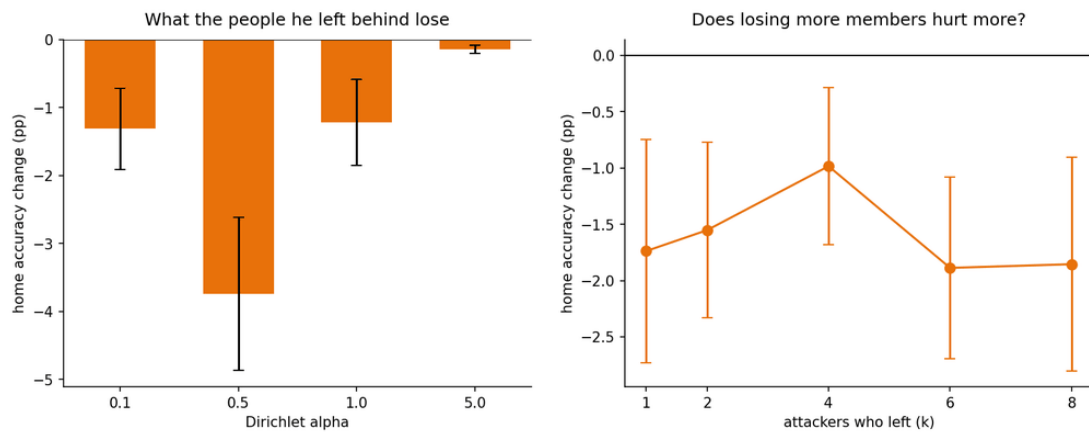


Fig. 19. MNIST: what the abandoned home cluster loses.

what does the cluster he abandoned lose?
-0.41 pp, standard error 0.14, on the home cluster's own distribution over 100 paired cells. The attack has a second victim that no measurement so far had looked at.

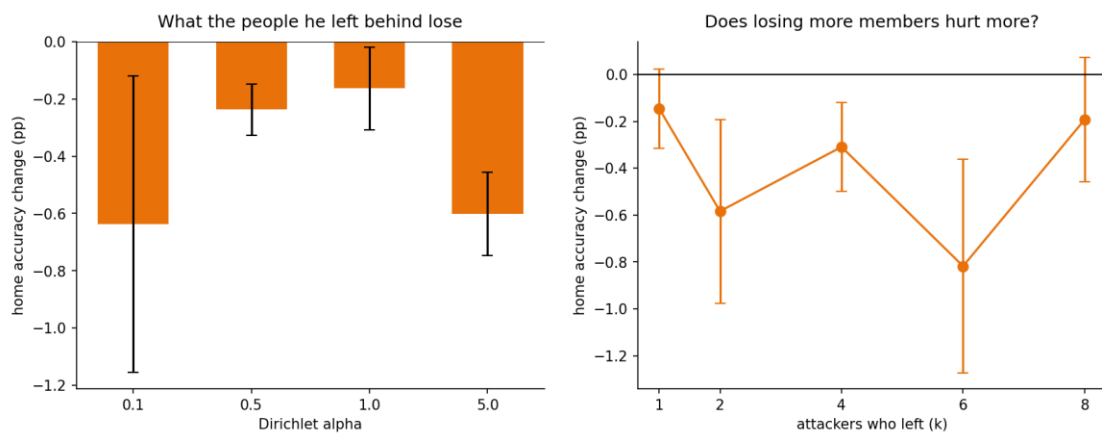


Fig. 20. MNIST: what the abandoned home cluster loses.

On MNIST the same growth (6.58 to 10.44) produces -0.12pp , not significant, because the models already sit near their ceiling. A monitor watching cluster accuracy would read a successful infiltration as a healthy federation.

The cluster he leaves is the real victim, and it is the one utility cost that replicates: -1.03 pp ($t = -3.04$) on IDS and -0.61 pp ($t = -2.84$) on MNIST.

Table 5. Utility effects against the truthful control, per seed, $df = 9$.

Effect	IDS-IoT	MNIST
attacker's own gain	-2.49 pp, $t = -3.13$	-1.06 pp, $t = -2.17$, not significant
victim cluster	+1.28 pp, $t = +2.76$	-0.12 pp, not significant
abandoned home cluster	-1.03 pp, $t = -3.04$	-0.61 pp, $t = -2.84$

Infiltration therefore costs the attacker accuracy rather than earning it.

Is the lie visible

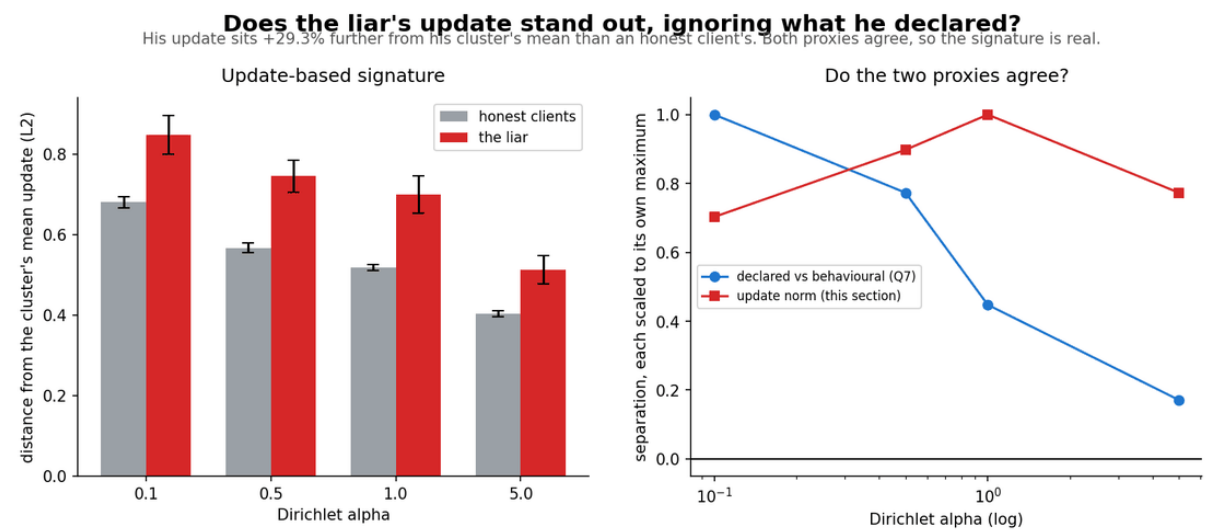


Fig. 21. IDS-IoT: the update-norm signature.

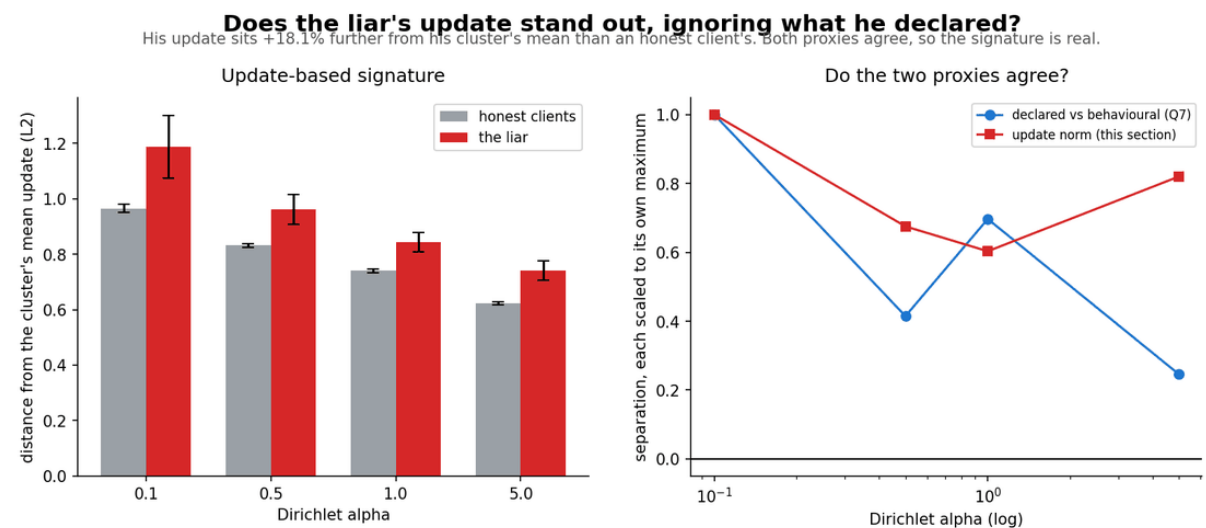


Fig. 22. MNIST: the update-norm signature.

As Fig. 8 shows, the liar's update sits $+31.3\% \pm 5.0$ further from its cluster mean than an honest client's on IDS, and $+15.8\% \pm 4.6$ on MNIST. A second proxy, comparing the declared histogram against the one his update implies, separates liar from honest by $+0.081$ and $+0.076$. Both fire, and the second does not look at his declaration at all. This is a signature, not a detector: no threshold, no false-positive rate, and every measurement here comes from a full-centroid declaration rather than the minimal lie that would suffice.

The declared sample count

What does overstating your sample count buy?

At 1000x he holds 99% of the aggregate and the cluster falls from 89.6% to 69.0%. He never moves cluster and never submits an abnormal update; only the weight on it is a lie.

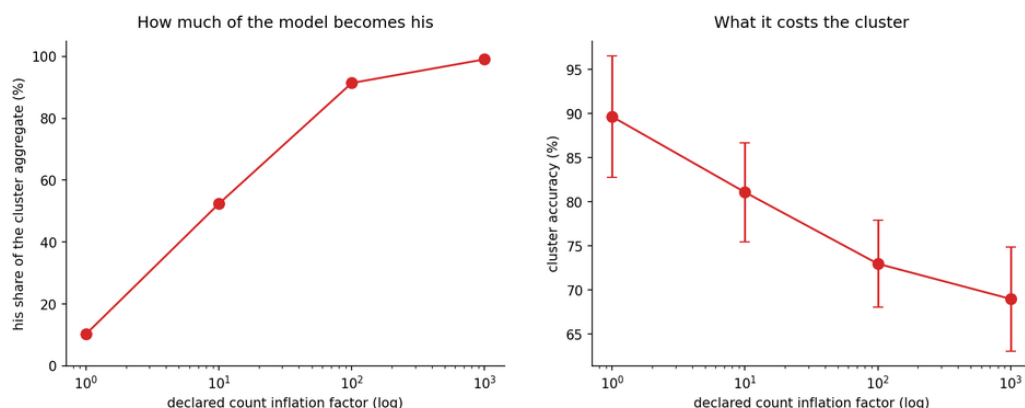


Fig. 23. IDS-IoT: the sample-count channel.

What does overstating your sample count buy?

At 1000x he holds 98% of the aggregate and the cluster falls from 91.4% to 76.0%. He never moves cluster and never submits an abnormal update; only the weight on it is a lie.

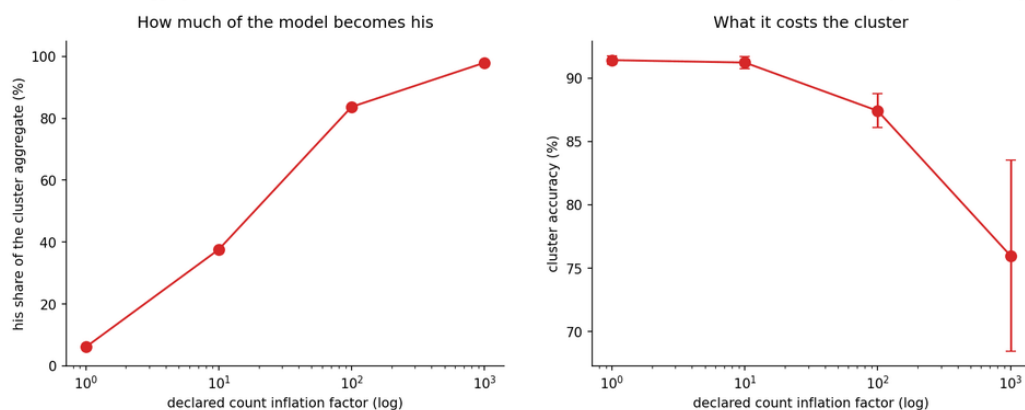


Fig. 24. MNIST: the sample-count channel.

Canonical FedAvg weights each update by a sample count the client reports and nobody verifies. One client inflating it a thousandfold (Fig. 9) takes $99.0\% \pm 0.1$ of its cluster's aggregate. It needs no infiltration and submits an honest update, so nothing inspecting update norms has anything to find. The accuracy cost is $+10.9 \pm 5.2$ pp on IDS, which is not significant, and $+13.8$ pp, $t = +3.64$ on MNIST, which is. This is the cheaper attack and the more damaging one.

Controls that changed the conclusions

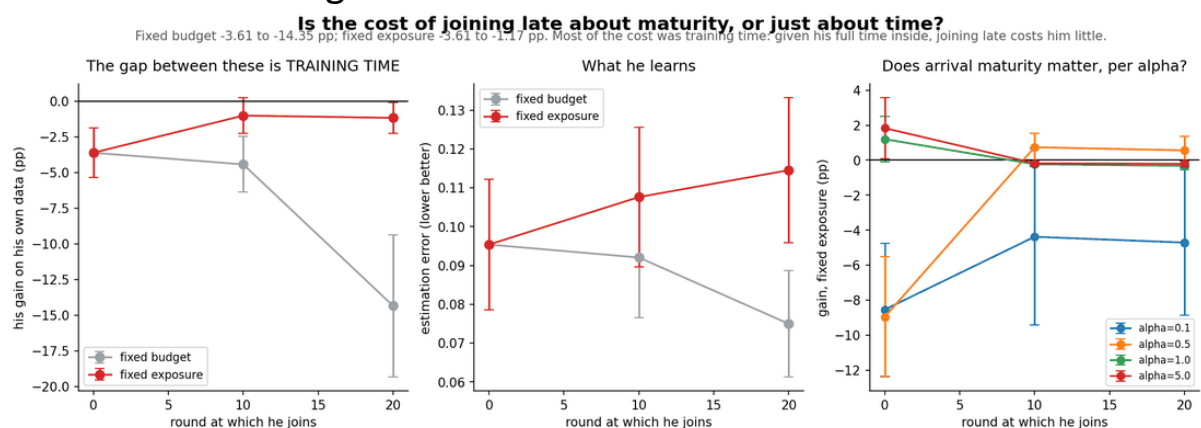


Fig. 25. IDS-IoT: fixed budget against fixed exposure.

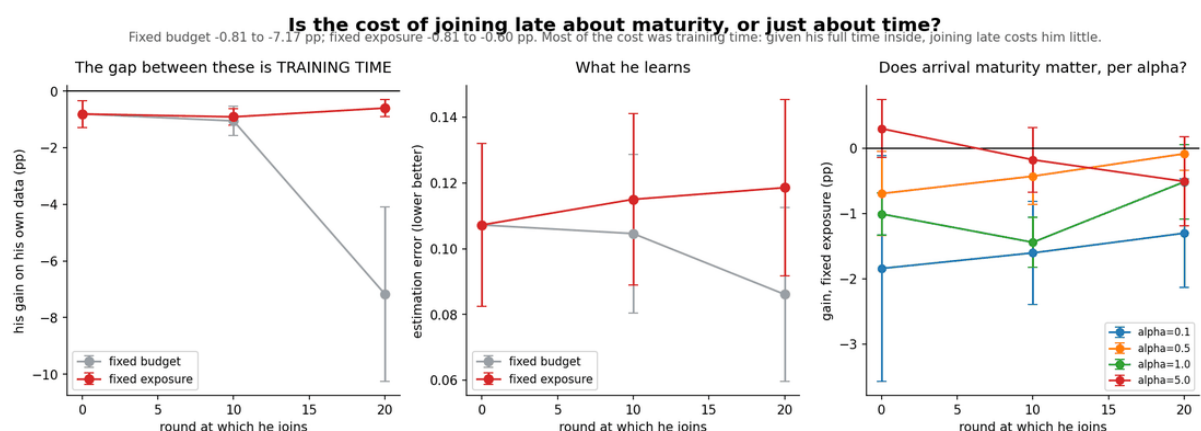


Fig. 26. MNIST: fixed budget against fixed exposure.

An attacker joining a converged cluster (Fig. 13) appears to lose heavily, -15.60 pp by round 20. Holding his exposure constant rather than the federation's budget removes it: -1.47 pp, and the trend vanishes ($p < 0.0001$ against $p = 0.941$). The apparent maturity penalty was training time.

Two further corrections matter. Doubling from five seeds to ten overturned two results: the victim-cluster effect became significant, and the sample-count damage became non-significant on IDS. And simultaneous raids interfere rather than compound: four at once do 2.31x the damage of one on IDS and 2.12x on MNIST, measured on the partition (ARI), where addition would give 4x.

Discussion

Neither test real client structure, both are centrally stored and Dirichlet partitioned, a writer-level dataset such as FEMNIST would be next. They have one small dense architecture each. There was no detector construction, so there is no claimed false positive rate. The evaluation of Byzantine-robust aggregation was not done, as separate work in this project has shown that only six rules affect the infiltration under this type of attack and partitioner. Section 2 (only) estimates the distribution of labels, not membership, feature distributions, or individual records. Both datasets can be clustered in an unrecognized way using declared label histograms, and in 20% of federations, the resulting partition is worse than random. The attack is not consistently superior to the model it is attacking,

and does not outperform a no-attack, reconnaissance baseline. It is best used in the cluster where the attacker leaves it, and it has its clearest system-level impact of corrupting the attacker's own perception of his cluster's population.

Conclusions

<< Draw conclusions from your complete project., include your view for the possible next steps of the work >>

REFERENCES

<< List references cited in the report following LNCS and/or UWE Harvard Referencing style>>

Peng, Z., Li, S., Chen, G., Zhang, C., Zhu, H. and Xue, M. (2022) 'Fingerprinting deep neural networks globally via universal adversarial perturbations', in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*. New Orleans, LA, 18-24 June 2022. Piscataway, NJ: IEEE, pp. 13430-13439.

Shuvo, M.S.R., Hossain, M.D., Mallik, A., Twigg, D. and Dagefu, F.T. (2025) *FLARE: a wireless side-channel fingerprinting attack on federated learning*. arXiv:2512.10296. Available at: <https://arxiv.org/abs/2512.10296> (Accessed: 25 June 2026).

Tariq, N. et al. (2025) 'Fingerprinting deep learning models via network traffic patterns in federated learning', in *Proceedings of the 2025 ACM workshop on wireless security and machine learning (WiseML)*. Arlington, VA, 3 July 2025. New York: ACM. Available at: <https://arxiv.org/abs/2506.03207> (Accessed: 25 June 2026).

Xiang, W. et al. (2018) *Verification for machine learning, autonomy, and neural networks survey*. arXiv:1810.01989. Available at: <https://arxiv.org/abs/1810.01989> (Accessed: 25 June 2026).

Ateniese, G., Mancini, L.V., Spognardi, A., Villani, A., Vitali, D. and Felici, G. (2015) 'Hacking smart machines with smarter ones: how to extract meaningful data from machine learning classifiers', *International Journal of Security and Networks*, 10(3), pp. 137–150.

Batina, L., Bhasin, S., Jap, D. and Picek, S. (2019) 'CSI NN: reverse engineering of neural network architectures through electromagnetic side channel'. In: *28th USENIX Security Symposium*, Santa Clara, CA, 14–16 August 2019. USENIX Association, pp. 515–532.

Cao, X., Jia, J. and Gong, N.Z. (2021) 'IPGuard: protecting intellectual property of deep neural networks via fingerprinting the classification boundary'. In: *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security (ASIA CCS '21)*, Hong Kong, 7–11 June 2021. New York: ACM, pp. 14–25.

Ganju, K., Wang, Q., Yang, W., Gunter, C.A. and Borisov, N. (2018) 'Property inference attacks on fully connected neural networks using permutation invariant representations'. In: *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS '18)*, Toronto, Canada, 15–19 October 2018. New York: ACM, pp. 619–633.

Gao, Y., Qiu, H., Zhang, Z., Wang, B., Ma, H., Abuadbbba, A., Xue, M., Fu, A. and Nepal, S. (2024) 'DeepTheft: stealing DNN model architectures through power side channel'. In: *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, pp. 3311–3326.

Hu, X., Liang, L., Li, S., Deng, L., Zuo, P., Ji, Y., Xie, X., Ding, Y., Liu, C., Sherwood, T. and Xie, Y. (2020) 'DeepSniffer: a DNN model extraction framework based on learning architectural hints'. In: Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '20), Lausanne, Switzerland, 16–20 March 2020. New York: ACM, pp. 385–399.

Oh, S.J., Augustin, M., Fritz, M. and Schiele, B. (2018) 'Towards reverse-engineering black-box neural networks'. In: 6th International Conference on Learning Representations (ICLR 2018), Vancouver, Canada, 30 April–3 May 2018.

Peng, Z., Li, S., Chen, G., Zhang, C., Zhu, H. and Xue, M. (2022) 'Fingerprinting deep neural networks globally via universal adversarial perturbations'. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), New Orleans, LA, 18–24 June 2022. IEEE, pp. 13430–13439. [pages unverified]

Rolnick, D. and Kording, K. (2020) 'Reverse-engineering deep ReLU networks'. In: Proceedings of the 37th International Conference on Machine Learning (ICML), 13–18 July 2020. PMLR 119, pp. 8178–8187.

Chen, J., Wang, J., Peng, T., Sun, Y., Cheng, P., Ji, S., Ma, X., Li, B. and Song, D. (2022) 'Copy, right? A testing framework for copyright protection of deep learning models'. In: 2022 IEEE Symposium on Security and Privacy (SP), San Francisco, CA, 22–26 May 2022. IEEE, pp. 824–841.

Shuvo, M.N.H. and Hossain, M. (2025) 'Fingerprinting deep learning models via network traffic patterns in federated learning'. In: Proceedings of the 2025 ACM Workshop on Wireless Security and Machine Learning (WiseML 2025), Arlington, VA, 3 July 2025. New York: ACM.

Shuvo, M.N.H., Hossain, M., Mallik, A., Twigg, J. and Dagefu, F. (2026) 'FLARE: a wireless side-channel fingerprinting attack on federated learning'. In: IEEE INFOCOM 2026 – IEEE Conference on Computer Communications. IEEE. Available from: <https://arxiv.org/abs/2512.10296> [Accessed 17 July 2026].

Yan, M., Fletcher, C.W. and Torrellas, J. (2020) 'Cache telepathy: leveraging shared resource attacks to learn DNN architectures'. In: 29th USENIX Security Symposium (USENIX Security 20), 12–14 August 2020. USENIX Association, pp. 2003–2020.

Bagdasaryan, E., Veit, A., Hua, K., Marcululescu, R., & Estrin, D. (2020). How to backdoor federated learning. International Conference on Artificial Intelligence and Statistics (AISTATS), 2938-2948.

Bhagoji, A. N., Chakraborty, S., Mittal, P., & Calo, S. (2019). Analyzing federated learning through an adversarial lens. International Conference on Machine Learning (ICML), 634-643.

Li, T., Sahu, A. K., Talwalkar, A., & Smith, V. (2020). Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine*, 37(3), 50-60.

Sattler, F., Müller, K. R., & Samek, W. (2020). Clustered federated learning: Model-Agnostic distributed learning from non-IID data. *IEEE Transactions on Neural Networks and Learning Systems*, 32(8), 3710-3722.

APPENDIX

<<All related supplementary details such secondary descriptions and details>>

Appendix A

<< [Provide the GitHub link of your code](#) >>

Appendix B

<< Insert all modelling diagrams in this section. Make sure each figure has a figure title with which you cite each within the text body >>